Nvidia AGX

GPU	512 核 Volta GPU with Tensor Cores
CPU	8 核 ARM v8.2 64 位 CPU, 8MB L2 + 4MB L3
内存	32GB 256 位 LPDDR4x | 137GB/s

Jetson AGX Xavier 配置指南

> 更新日期: 2020年06月04日
>
> 文档作者: 张栋祥
>
> 联系邮箱: zzudongxiang@163.com

1. 系统镜像烧录

准备工作:

A. 一台 `Ubuntu` 系统的电脑, 要可以使用USB接口

B. 一台全新的 `Jetson AGX Xavier` 设备(下称 `AGX` 或 `Jetson`)或待重装系统的 `AGX` 设备

C. 一个USB Hub, 可以接到 `AGX` 上, 同时需要外接的鼠标, 键盘

D. 一台显示器, 如果是VGA接口需要一个HDMI转VGA的转接器

E. (非必要)一个局域网环境, 最好使用有线网络连接 `AGX`

F. `AGX` 对应的电源线, USB线等相关线材(在 `AGX` 的包装盒内)

准备 Nvidia SDK Manager

在官方网站下载 `Nvidia SDK Manager` (下称 `SDKM`)

https://developer.nvidia.com/nvidia-sdk-manager

注册Nvidia开发者账号并下载SDKM到Ubuntu设备上

```
$ sudo apt install ./sdkmanager_X.X.X-XXXX_amd64.deb
# 其中的版本号应根据下载的文件确定, 使用时可以使用tab补全文件名
```

烧录前准备

`AGX` 请勿接入电源

A. `AGX` 带灯一侧的Type3接口连接到Ubuntu电脑上

B. `AGX` 电源一侧的Type3接口链接USB Hub, 再接上键盘, 鼠标待用

C. `AGX` 电源一侧的网络接口接入局域网络(路由器或交换机)

D. `AGX` 电源一侧的HDMI接口接入显示器

E. `AGX` 电源一侧的SD卡槽不插任何卡, 默认留空即可

F. 其他未说明接口均保持原状

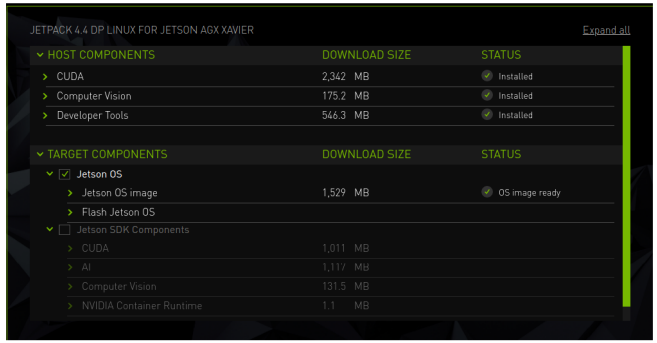
烧录过程

注意: 安装 `SDKM` 的Ubuntu设备最好可以访问外网, 防止文件下载失败

保持AGX电源断开状态, 打开 `SDKM` 软件, 使用Nvidia开发者账号登陆

STEP1 保持默认即可, 点击 `继续` 到STEP2

STEP2 中选择 `Jetson OS` 及其下属所有选项, 取消勾选 `Jetson SDK Components` 及其下属所有选项



STEP3 输入Ubuntu的管理员密码后继续, 等待安装完成

待 `SDKM` 提示找不到USB设备时, 插上 `AGX` 设备电源

进入Recovery模式:

a). 按着 Power 键不放

b). 按着电源键旁边的 Force Recovery 键不放

c). 轻按一下最旁边的 Reset 键

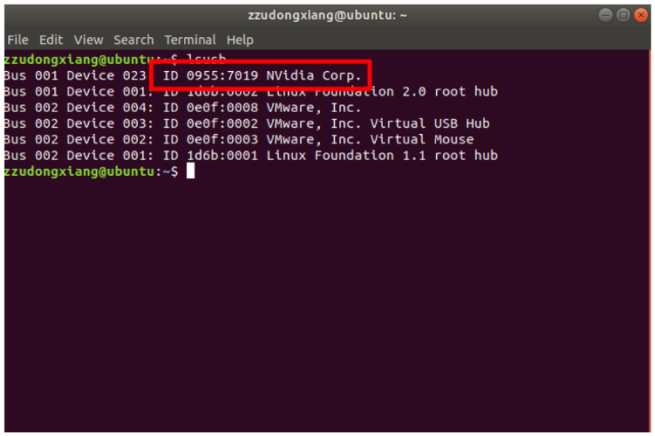
d). 等2~5秒后同时松开 Power 键与 Force Recovery 键

e). 注意整个过程不要超过10秒, 按下 Power 键10秒为强制关机, 无法进入Recovery 模式

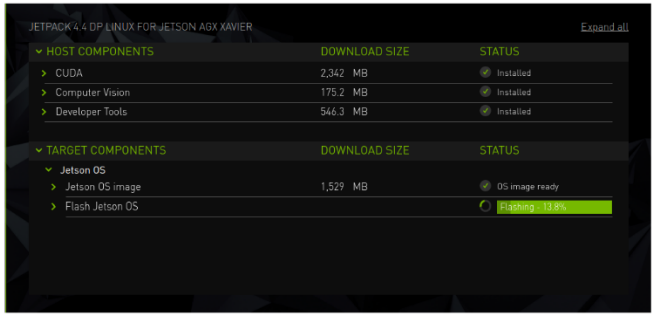
在安装 `SDKM` 的Ubuntu控制台输入

```
$ lsusb
```

如果出现 Nvidia Crop 的设备说明 `AGX` 成功进入 Recovery 模式



然后恢复软件的运行



如果经过此步骤依旧无法继续烧录, 请多次重试

烧录完成之后重启设备(拔掉电源线重新插, 插好电源后点击 Power 键开机即可)

在 `AGX` 的界面完成安装, 安装中建议使用 `英文版本` 的系统

2. 更新系统, 并开启风扇

> 注意: 一定不要换源 !!!

执行更新程序

```
$ sudo apt update
$ sudo apt upgrade
```

删除用户密码(方便开机启动风扇)

```
$ sudo su
$ sudo nano /etc/sudoers
```

在文档的最后添加上

```
<your_name> ALL=(ALL) NOPASSWD:ALL
例如:
jetson ALL=(ALL) NOPASSWD:ALL
```

同时将以下内容进行修改

```
%admin ALL=(ALL) ALL
改为:
%admin ALL=(ALL) NOPASSWD:ALL
```

然后使用 `exit` 命令退出root用户

开启风扇

在更新系统, 重启设备之后会默认安装风扇驱动管理程序, 使用以下命令开启风扇

```
$ sudo jetson_clocks
```

如果听到风扇开始正常运转则启动成功

开启 AGX 全功率运行

```
$ sudo nvpmodel -m 3
# 可使用 sudo nvpmodel -q 查看当前运行功率
```

3. 配置Python与pip环境

经过上一步, 使用外接显示屏完成系统安装后可以使用ssh与 `AGX` 进行连接了

安装 Python3-dev 与 Python3-pip

```
$ sudo apt install python3-dev
# 可以通过以下的源码方式安装pip
$ sudo apt install python3-pip
```

※: 从源码安装 pip

建议从源码安装pip, 可以避免后期更新pip或pip安装出现问题

```
# 提前从源码安装setuptools工具
# setuptools 官方网站 https://pypi.org/project/setuptools/#files
$ wget
https://files.pythonhosted.org/packages/25/bf/a37e89d3148221fedd4def62bb68749041
d79f3840d58a7943f81a6f6c6e/setuptools-47.1.1.zip
$ sudo unzip setuptools-47.1.1.zip
$ sudo cd setuptools-47.1.1
$ sudo python3 setup.py install
# 删除安装文件目录
$ cd ../
$ sudo rm -rf setuptools-47.1.1
$ sudo rm -rf setuptools-47.1.1.zip
```

下载pip源码并安装

```
# pip 官方网站 https://pypi.org/project/pip/#files
$ wget
https://files.pythonhosted.org/packages/08/25/f204a6138dade2f6757b4ae99bc3994aac
28a5602c97ddb2a35e0e22fbc4/pip-20.1.1.tar.gz
$ sudo tar -xzvf pip-20.1.1.tar.gz
$ cd pip-20.1.1
$ sudo python3 setup.py build
$ sudo python3 setup.py install
# 删除安装文件目录
$ cd ../
$ sudo rm -rf pip-20.1.1
$ sudo rm -rf pip-20.1.1.tar.gz
```

查看pip3版本

```
$ pip3 -v
# pip 20.1.1 from /usr/local/lib/python3.6/dist-packages...
# 可以考虑升级pip版本，命令如下
$ sudo python3 -m pip install --upgrade pip
```

4. 安装与配置 Jupyter

```
$ sudo pip3 install jupyter
# 如果下载失败，可以使用其他源进行安装，例如使用以下命令
$ sudo pip3 install jupyter -i https://pypi.tuna.tsinghua.edu.cn/simple
```

> 在安装过程中会在 `Running setup.py install for pyzmq ...` 消耗很长时间，请耐心等待

配置 Jupyter

```
# 生成一个Jupyter的配置文件
$ jupyter-notebook --generate-config
# 设置Jupyter的登陆密码
$ jupyter-notebook password
# 修改IP，端口等配置
$ sudo nano ~/.jupyter/jupyter_notebook_config.py
# 取消以下行注释，并修改对应的参数值
c.NotebookApp.open_browser = False
c.NotebookApp.ip = '0.0.0.0'
c.NotebookApp.port = 80
c.NotebookApp.allow_root = True
```

安装 Jupyter 插件 nbextensions

```
# 安装 jupyter_contrib_nbextensions
$ sudo pip3 install jupyter_contrib_nbextensions
$ sudo jupyter contrib nbextension install
# 安装 jupyter_nbextensions_configurator
$ sudo pip3 install jupyter_nbextensions_configurator
$ jupyter nbextensions_configurator enable
```

测试启动 Jupyter

```
$ sudo jupyter-notebook
```

附注

> 在后续使用pip安装时可以使用以下命令进行安装

```
$ sudo python3 -m pip install <package_name>
# 例如:
$ sudo python3 -m pip install matplotlib -i
https://pypi.tuna.tsinghua.edu.cn/simple
```

> 如果pip不是最新版的，使用此方法可避免出现警告

5. 配置AXG远程桌面连接

解决 Desktop Shareing 崩溃问题

```
$ sudo nano /usr/share/glib-2.0/schemas/org.gnome.Vino.gschema.xml
```

将以下内容粘贴到对应的文件中

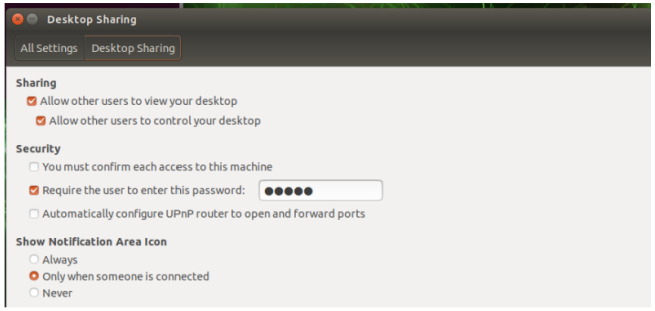
```
<key name='enabled' type='b'>
    <summary>Enable remote access to the desktop</summary>
    <description>
        If true, allows remote access to the desktop via the RFB
        protocol. Users on remote machines may then connect to the
        desktop using a VNC viewer.
    </description>
    <default>true</default>
</key>
```

重新编译Gnome架构

```
$ sudo glib-compile-schemas /usr/share/glib-2.0/schemas
```

选择 Allow other user to view your desktop 选项

选择 Require the user to enter this password 选项，并配置一个密码



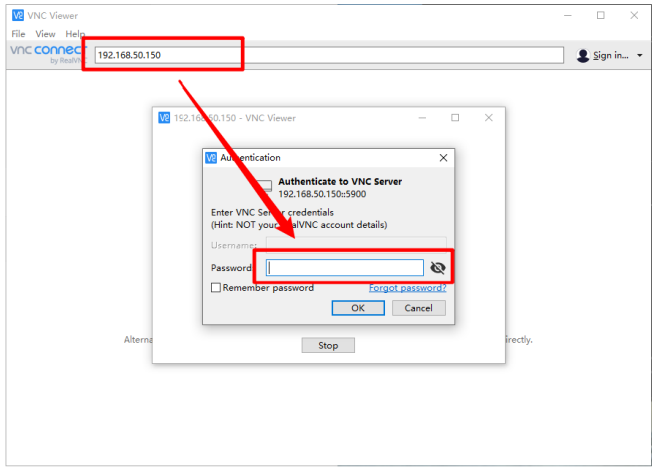
禁用VNC远程连接加密

```
$ gsettings set org.gnome.Vino require-encryption false
$ gsettings set org.gnome.Vino prompt-enabled false
```

启动 VNC 远程连接

```
# 该操作无法在ssh控制台远程完成
$ sudo /usr/lib/vino/vino-server
```

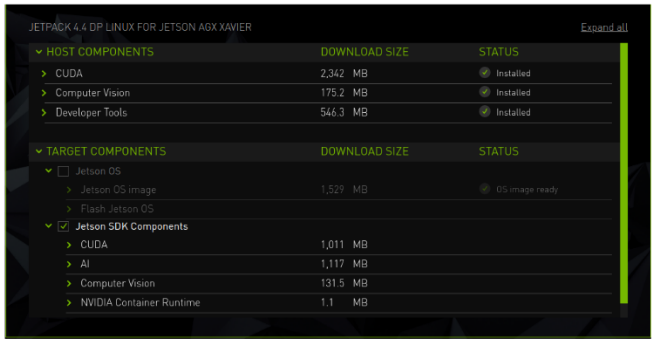
使用 VNC 远程连接进行测试



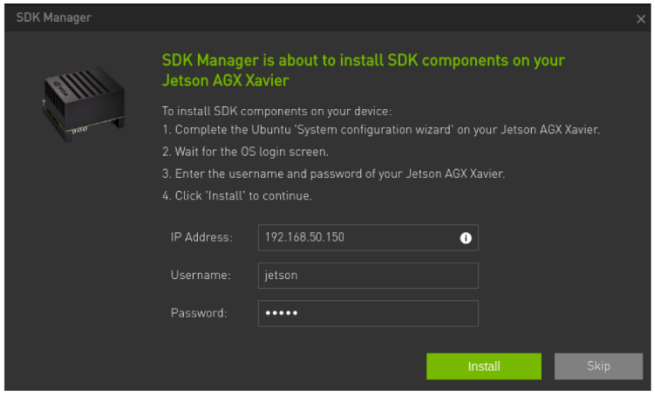
输入在 Require the user to enter this password 中设置的密码进行连接测试

6. 安装 SDK Components

打开 `SDKM` 软件, 按照系统烧录的教程, 在STEP2中取消勾选 Jetson OS 同时勾选 Jetson SDK Components



等待下载文件, 下载完成开始安装



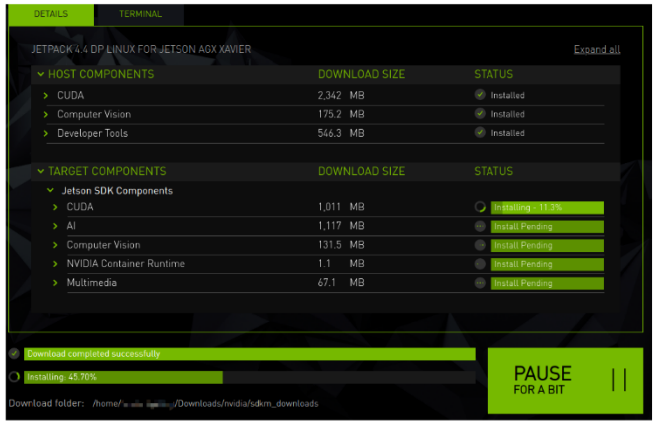
`IP` : 填写 `Jetson` 在局域网中的IP地址, 可通过ifconfig命令查看

`Username` : `Jetson` 系统中具有管理员权限的账户

`Password` : Username账户对应的密码

点击 Install开始安装SDK

> 该过程 `AGX` 需要全程开机并连接局域网



建议使用有线网络进行连接，已保持连接的稳定性

- 如果出现错误，请查看消息日志
- 中间可能会出现错误，请耐心多试几次

7. 设置开机任务

开机启动 VNC 远程连接

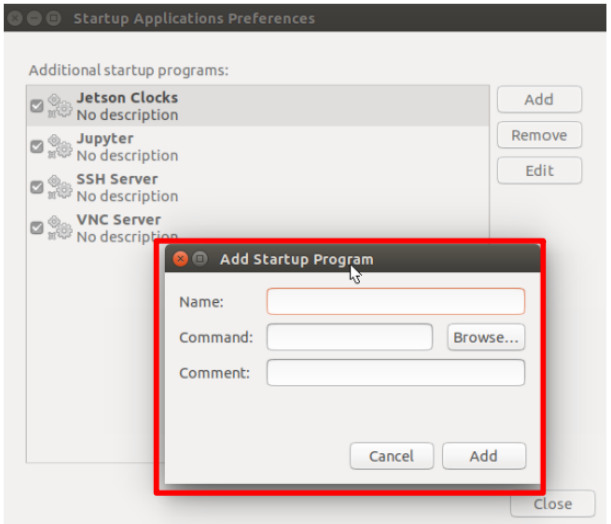
以下操作需要在 AGX 的显示器上面操作

点击开始菜单，启动 `Startup Applications` 应用，点击 `Add` 添加开机命令

`Name`：自由起名字，默认建议 VNC Server

`Command`：sudo nohup /usr/lib/vino/vino-server >> ~/vnc.log &

`Comment`：随便写入点什么都行，默认建议留空即可



开机加载 rc.local 脚本启动风扇

> 以上开机启动仅适用于图形化界面显示状态，对于命令行状态无效

```
$ sudo nano /lib/systemd/system/rc.local.service
# 如果不存在这个文件就创建新的文件
```

在这个文件后面添加

```
[Install]
WantedBy=multi-user.target
Alias=rc-local.service
```

因此这个文件的完整内容为(适用于新建该文件的时候)

```
[Unit]
Description=/etc/rc.local Compatibility
Documentation=man:systemd-rc-local-generator(8)
ConditionFileIsExecutable=/etc/rc.local
After=network.target

[Service]
Type=forking
```

```
ExecStart=/etc/rc.local start
TimeoutSec=0
RemainAfterExit=yes
GuessMainPID=no

[Install]
WantedBy=multi-user.target
Alias=rc-local.service
```

编辑开机启动脚本

```
# 如果该文件不存在则新建文件
$ sudo nano /etc/rc.local
```

添加开机启动命令

```
#!/bin/sh -e
#
# rc.local
#
# This script is executed at the end of each multiuser runlevel.
# Make sure that the script will "exit 0" on success or any other
# value on error.
#
# In order to enable or disable this script just change the execution
# bits.
#

# 开机启动风扇
sudo jetson_clocks

exit 0
```

给 rc.local 添加可执行权限

```
$ sudo chmod a+x /etc/rc.local
```

添加软链接, 挂载开机启动服务

```
$ sudo ln -s /lib/systemd/system/rc.local.service /etc/systemd/system/
```

开机启动Jupyter服务

新建 Jupyter 服务文件

```
$ sudo nano /lib/systemd/system/jupyter.service
```

将以下内容写入文件

```
[Unit]
Description=Jupyter Service
After=network.target
```

```
[Service]
User=jetson
Type=simple
ExecStart=~/.jupyter/service.sh
TimeoutSec=0
RemainAfterExit=yes
GuessMainPID=no

[Install]
WantedBy=multi-user.target
```

添加软链接, 挂载开机启动服务

```
$ sudo ln -s /lib/systemd/system/jupyter.service /etc/systemd/system/
```

编辑 Jupyter.service 写入开始执行的命令

```
$ sudo nano ~/.jupyter/service.sh
```

在文件中添加以下内容

```
#!/bin/sh -e
#
# rc.local
#
# This script is executed at the end of each multiuser runlevel.
# Make sure that the script will "exit 0" on success or any other
# value on error.
#
# In order to enable or disable this script just change the execution
# bits.
#

# 开机启动Jupyter
sudo nohup jupyter-notebook >> ~/.jupyter/jupyter.log &

exit 0
```

给 service.sh 脚本添加可执行权限

```
$ sudo chmod a+x ~/.jupyter/service.sh
```

开启服务

```
$ sudo systemctl enable jupyter
$ sudo systemctl start jupyter
```

8. 开启或关闭图形化显示界面

关闭图形化显示界面

```
$ sudo systemctl set-default multi-user.target
$ sudo reboot
```

开启图形化显示界面

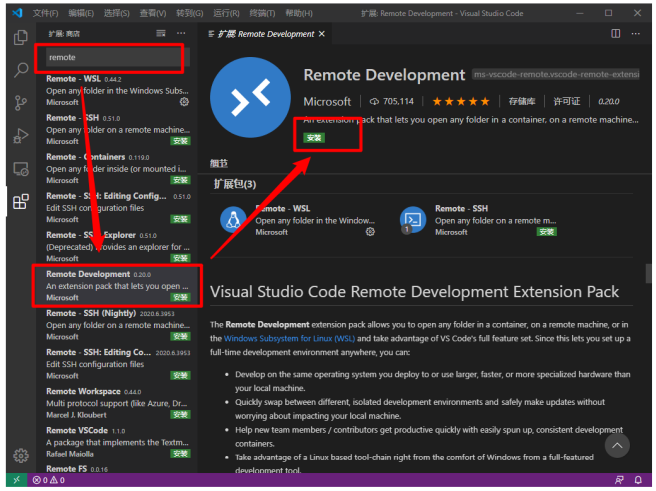
```
$ sudo systemctl set-default graphical.target
$ sudo reboot
```

9. 使用方式

使用SSH连接设备

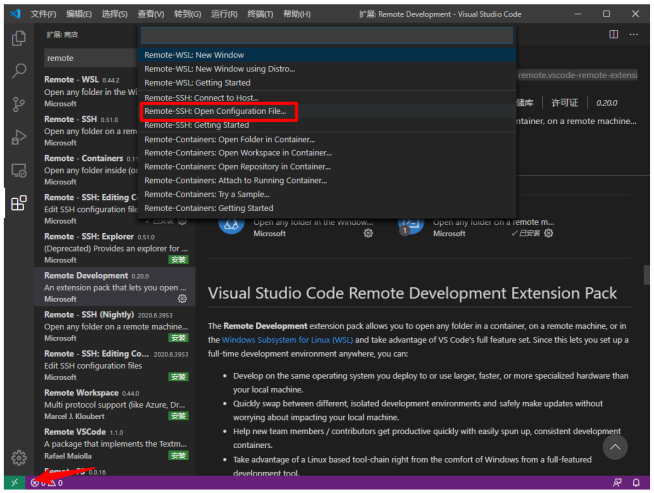
在 VSCode 中搜索 `Remote Development` 插件并进行安装

配置 ssh 连接, 使用 VSCode 直接操作 `AGX` 设备



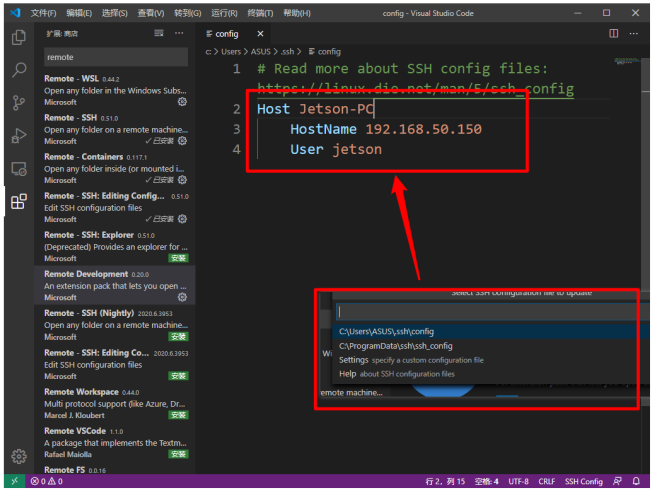
> 安装 `Remote Development` 时会自动安装相关的插件

安装完成后点击左下角绿色图标进行连接, 选择 `Open Configuration File ...` 选项



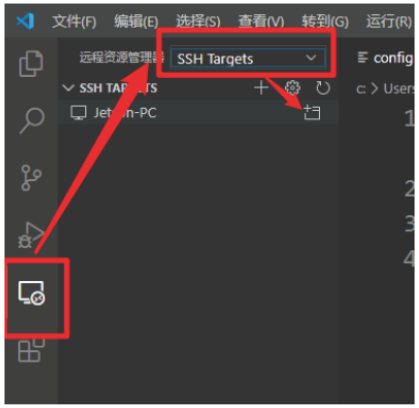
选择一个SSH配置文件保存的位置, 并修改配置文件内容

> Host: SSH连接的名字, 可以自由取
>
> HostName: SSH连接的地址
>
> User: 登录SSH连接的用户名
>
> SSH连接的密码会在连接时提示用户输入



此时可以使用左侧面板中的 `远程资源管理器` 连接 `AGX` 设备

> 注意远程资源管理器中要选中 `SSH Targets`



使用Jupyter连接设备

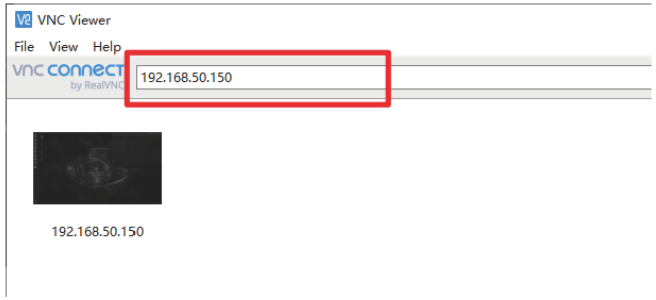
直接在浏览器中输入设备的IP地址即可打开jupyter页面

jupyter操作界面

使用VNC Viewer连接设备

下载 VNC Viewer 并在地址栏中输入VNC Viewer的IP地址进行连接

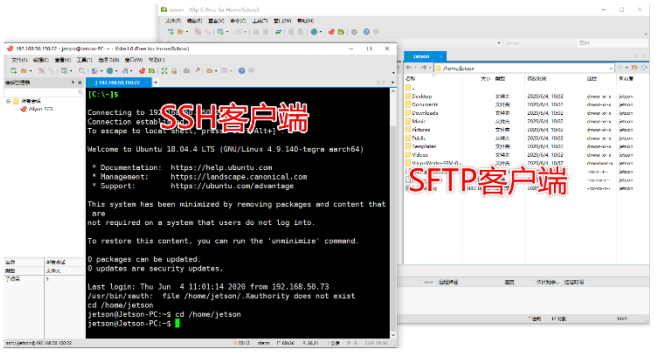
VNC Viewer下载地址: https://www.realvnc.com/en/connect/download/viewer/



使用 SSH/SFTP 客户端连接

使用SSH客户端连接设备, 或使用SFTP客户端连接设备

可以使用SFTP客户端与设备之间传输文件



直接使用显示屏/键鼠操作设备

按照规定连接好相关电缆即可